

Instituto Superior de Engenharia de Lisboa
Licenciatura em Engenharia Informática e de Computadores
Programação Concorrente
Exame de Época Normal, Verão de 2023/2024
Duração: 3 horas

1. [4] Considere a classe **UnsafeCyclicSuccession**, que tem por objectivo representar uma sequência de elementos, consumidos de forma sequencial e cíclica através do método **next**. Assuma que os elementos do *array* não são alterados após a construção da instância da sequência.

```
class UnsafeCyclicSuccession<T>(  
    private val items: Array<T>  
) {  
    private var index = 0  
    fun next(): T {  
        val res = items[index]  
        index  
        if(index == items.size) index = 0  
        return res  
    }  
}
```

+=

1

- a. [1,5] Realize uma versão *thread-safe* desta classe, com recurso ao uso de *locks*.
b. [2,5] Realize uma versão *thread-safe* desta classe, sem recurso ao uso de *locks*.
2. [4] Realize o sincronizador **CyclicCountDownLatch** com a interface apresentada em seguida.

```
class CyclicCountDownLatch(val initialCount: Int) {  
    init { require(initialCount > 0) }  
  
    fun countDown(): Int { ... }  
  
    @Throws(InterruptedException::class)  
    fun await(timeout: Duration): Boolean { ... }  
}
```

A função **countDown** decrementa o contador. Se o contador chegar a zero, o contador deve ser repostado de imediato com **initialCount** e todas as *threads* que estejam nesse momento em espera em **await** devem terminar a espera com sucesso. O retorno de uma chamada à função **countDown** indica o número de *threads* que terminaram a espera com sucesso devido a essa chamada.

A função **await** deve esperar, de forma passiva, que uma chamada à função **countDown** decremente o contador para o valor de zero, aceitando cancelamento por *timeout* ou por interrupção da *thread*. A função **await** retorna **true** caso a espera termine com sucesso, ou **false** caso a espera termine por *timeout*.

3. [4] Realize o sincronizador **UnboundedMessageQueue** com a interface apresentada em seguida.

```
class UnboundedMessageQueue<T> {  
    @Throws(RejectedExecutionException::class)  
    fun enqueue(value: T): Unit { ... }  
    @Throws(InterruptedException::class)  
    fun tryDequeue(timeout: Duration): T? { ... }  
    @Throws(InterruptedException::class)  
    fun closeAndWaitForEmpty(timeout: Duration): Boolean { ... }  
}
```

A função **enqueue** entrega uma mensagem à fila, que tem capacidade ilimitada. A função **tryDequeue** remove e retorna uma mensagem da fila, ficando bloqueada enquanto o pedido não puder ser satisfeito. A comunicação deve usar o critério FIFO (*first in first out*): dadas duas mensagens colocadas na fila, a primeira a ser entregue a um consumidor deve ser a primeira que foi entregue à fila; caso existam dois ou mais consumidores à espera de uma mensagem, o primeiro a ver o seu pedido satisfeito é o que está à espera há mais tempo. A função **closeAndWaitForEmpty** coloca a fila em modo de encerramento e espera que esta fique sem mensagens. Neste modo, a fila já não aceita inserção de novas mensagens e qualquer chamada a **enqueue** deve terminar com o lançamento da exceção **RejectedExecutionException**.

As funções **dequeue** e **closeAndWaitForEmpty** aceitam como parâmetro o tempo máximo de espera e devem ser sensíveis a interrupções, tratando-as de acordo com o protocolo definido na plataforma Java. Note que uma chamada a **tryDequeue** deve terminar com insucesso se a fila estiver encerrada e não existirem mensagens nela presentes.

4. [4] Realize o sincronizador **MessageSender** com a interface apresentada em seguida.

```
class MessageSender<T> {  
    suspend fun waitForMessage(): T { ... }  
    suspend fun sendToOne(message: T): Boolean { ... }  
}
```

A função **waitForMessage** suspende a corrotina onde foi realizada a invocação até que uma mensagem lhe seja entregue através da função **sendToOne**. A função **sendToOne** entrega a mensagem à primeira das corrotinas em espera na função **waitForMessage**, retornando **true** caso a mensagem seja entregue ou **false** caso não existam corrotinas em espera nesse momento. A mensagem passada na chamada **sendToOne** não deve ficar disponível para chamadas futuras da função **waitForMessage**. Não é necessário suportar cancelamento.

5. [4] Realize a função

```
suspend fun <T> either(f1: CompletableFuture<T>, f2: CompletableFuture<T>): T
```

Esta função retorna o resultado do primeiro *future* a completar-se com sucesso, ou lança uma exceção contendo os erros com que ambos os *futures* se completaram. Não é necessário suportar cancelamento.

Duração: 3 horas
ISEL, 21 de junho de 2024